

CURSO DE JAVA

CAPÍTULO 2

SALIDAS EN JAVA

Desde principiante hasta nivel práctico

Nivel	Desde principiante hasta nivel práctico
Enfoque	Desarrollo de Java
Tipo	Capítulo técnico y educativo
Objetivo	Comprender y aplicar las salidas de consola en Java

Material diseñado para estudio autónomo, práctica progresiva y aplicación en sistemas reales.

Objetivo del capítulo

La salida de información es la forma en que un programa comunica datos y resultados. En Java, la salida de consola se realiza principalmente mediante `System.out` y sus métodos `print()`, `println()` y `printf()`. Aprenderás desde mostrar un mensaje sencillo hasta construir reportes claros y formateados.

Este capítulo no busca que memorices instrucciones sin comprenderlas. Busca que puedas explicar qué ocurre, elegir el método apropiado, controlar el formato y detectar errores.

Al finalizar este capítulo serás capaz de:

- Explicar qué es una salida y por qué un programa necesita comunicar información.
- Identificar `System`, `out`, `print()`, `println()` y `printf()`.
- Mostrar texto y valores en la consola.
- Distinguir `print()` de `println()`.
- Usar concatenación y secuencias de escape.
- Usar `printf()` para controlar números, texto y decimales.
- Construir reportes de consola ordenados.
- Detectar errores frecuentes de salida.
- Aplicar salidas en ejemplos de máquinas, sensores, producción e inventarios.
- Construir una interfaz de consola básica para un sistema industrial.

1. Introducción a las salidas

Un programa puede recibir datos, procesarlos y producir resultados. La **salida** es la información que el programa envía hacia un destino. Cuando trabajamos con programas de consola, normalmente ese destino es la salida estándar que observamos en la terminal.

Analogía industrial: un sensor puede producir una temperatura, un controlador puede procesarla y un panel puede mostrar el estado al operador. Una aplicación Java puede hacer algo conceptualmente similar: obtiene o genera datos, los procesa y los presenta.

La salida existe para que podamos observar resultados, informar estados, presentar reportes, confirmar operaciones y diagnosticar problemas. Sin salida, aprender y probar un programa sería mucho más difícil.

En sistemas profesionales, la información puede terminar en una consola, archivo, registro, interfaz gráfica, red, base de datos o API. Aquí nos enfocaremos en la consola porque es una de las primeras herramientas para aprender programación.

2. Comprendiendo `System.out`

Antes de estudiar los métodos de salida conviene entender que **`System.out`** no es una palabra única. Es una expresión formada por diferentes componentes.

Parte	Función básica	Explicación sencilla
<code>System</code>	Clase de Java	Proporciona funcionalidades relacionadas con el entorno del programa.
<code>out</code>	Salida estándar	Representa el flujo de salida habitual de la consola.

Parte	Función básica	Explicación sencilla
print()	Método de salida	Escribe información sin salto automático.
println()	Método de salida	Escribe información y agrega un salto de línea.
printf()	Método de salida formateada	Escribe usando una plantilla de formato.

Por ahora puedes interpretar `System.out.println("Hola");` como: *usar la salida estándar del sistema para escribir Hola y terminar la línea.*

3. print(): salida sin salto automático

¿Qué es? `print()` escribe contenido en la salida estándar y deja la siguiente salida en la misma línea.

¿Por qué se utiliza? Permite construir una línea por partes y controlar manualmente dónde se produce el salto.

¿Cuándo? Cuando varias piezas de información deben permanecer en una misma línea.

Sintaxis:

```
System.out.print("texto");
```

Ejemplo básico:

```
public class Main {
    public static void main(String[] args) {
        System.out.print("Hola ");
        System.out.print("Java");
    }
}
```

Línea por línea: la clase `Main` contiene `main`; la primera llamada imprime "Hola " y no cambia de línea; la segunda imprime "Java" justo después.

Resultado: Hola Java en una sola línea.

4. println(): salida con salto de línea

¿Qué es? `println()` escribe contenido y después agrega un salto de línea.

¿Por qué? Facilita presentar mensajes independientes y reportes donde cada dato debe aparecer en una línea.

Sintaxis:

```
System.out.println("texto");
```

Ejemplo:

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Máquina iniciada");
        System.out.println("Sensor activo");
        System.out.println("Sistema listo");
    }
}
```

Cada llamada genera una línea. El resultado es un pequeño reporte vertical.

5. print() vs println()

Característica	print()	println()
Escribe contenido	Sí	Sí
Salto automático	No	Sí
Uso típico	Construir una línea por partes	Mostrar líneas independientes
Ejemplo	print("A");	println("A");

Ejemplo de análisis:

```
System.out.print("A");
System.out.print("B");
System.out.println("C");
System.out.println("D");
```

Resultado: **ABC** queda en la primera línea y **D** en la segunda. El salto ocurre después de C.

6. Texto y String en las salidas

Un **String** representa texto. Los literales de texto se escriben entre comillas dobles. La salida puede recibir ese texto directamente.

```
System.out.println("Temperatura normal");
System.out.println("Producción estable");
```

Las comillas delimitan el contenido textual. Un error de comillas puede provocar mensajes de compilación que parecen señalar lugares extraños, porque el compilador deja de interpretar correctamente el resto de la línea.

```
System.out.println("Sensor activo);
```

En este ejemplo falta la comilla de cierre.

7. Concatenación con +

La concatenación permite combinar texto y valores para construir un mensaje. El operador **+** puede actuar como suma numérica o como concatenación cuando interviene texto.

Ejemplo básico:

```
int piezas = 250;
System.out.println("Piezas producidas: " + piezas);
```

Java combina el texto con el valor de piezas para producir un mensaje que el usuario puede leer.

Ejemplo industrial:

```
int temperatura = 72;
int velocidad = 1500;

System.out.println("Temperatura: " + temperatura + " °C");
System.out.println("Velocidad: " + velocidad + " RPM");
```

Detalle importante: el orden importa cuando + mezcla números y texto.

```
System.out.println("Resultado: " + 10 + 5);
System.out.println(10 + 5 + " es el resultado");
```

La primera produce **Resultado: 105**; la segunda produce **15 es el resultado**. En la primera, la presencia inicial de texto hace que los **+** posteriores concatenen. En la segunda, **10 + 5** se evalúa antes de unirse al texto.

8. Secuencias de escape

Las secuencias de escape representan caracteres especiales dentro de una cadena. Comienzan con una barra invertida.

Secuencia	Significado	Ejemplo
<code>\n</code>	Salto de línea	"A\nB"
<code>\t</code>	Tabulación	"Código\tEstado"
<code>\"</code>	Comilla doble	"Dijo: \"OK\""
<code>\\</code>	Barra invertida	"C:\\Java"

```
System.out.println("=== REPORTE ===");
System.out.println("Código\tEstado");
System.out.println("M-01\tACTIVA");
System.out.println("M-02\tDETENIDA");
```

Las secuencias permiten mejorar la presentación sin crear una instrucción independiente para cada detalle.

9. printf(): salida con formato

`printf()` permite escribir usando una cadena de formato. Es especialmente útil cuando necesitas controlar cómo se muestran números, decimales, texto o reportes.

Sintaxis general:

```
System.out.printf("formato", valor1, valor2);
```

Formato	Uso	Ejemplo
<code>%s</code>	Texto	<code>printf("Máquina: %s", nombre)</code>
<code>%d</code>	Entero	<code>printf("Piezas: %d", piezas)</code>
<code>%f</code>	Decimal	<code>printf("Temp: %f", temperatura)</code>
<code>%.2f</code>	Decimal con 2 posiciones	<code>printf("Temp: %.2f", temperatura)</code>
<code>%n</code>	Salto de línea	<code>printf("OK%n")</code>
<code>%%</code>	Porcentaje literal	<code>printf("100%%")</code>

Ejemplo completo:

```
String maquina = "M-01";
int piezas = 850;
double temperatura = 72.4567;

System.out.printf("Máquina: %s\n", maquina);
System.out.printf("Piezas: %d\n", piezas);
System.out.printf("Temperatura: %.2f °C\n", temperatura);
```

Resultado aproximado: Máquina: M-01; Piezas: 850; Temperatura: 72.46 °C, cada dato en su propia línea.

El valor almacenado puede conservar más precisión; `%.2f` controla la forma en que se presenta.

10. Ejemplos progresivos

Nivel 1 — Ejemplo básico

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hola, Java");
    }
}
```

Sirve para aprender la estructura mínima de una salida. Los errores típicos son olvidar `;`, escribir mal `println` o no cerrar las comillas.

Nivel 2 — Ejemplo intermedio

```
public class Sistema {
    public static void main(String[] args) {
        System.out.println("=== SISTEMA ===");
        System.out.println("Estado: ACTIVO");
        System.out.println("Modo: AUTOMÁTICO");
        System.out.println("Producción: 500 piezas");
    }
}
```

Se construye un pequeño reporte utilizando varias instrucciones independientes.

Nivel 3 — Ejemplo práctico

```
public class Monitoreo {
    public static void main(String[] args) {
        String maquina = "M-01";
        double temperatura = 68.5;
        int piezas = 850;

        System.out.println("=== MONITOREO ===");
        System.out.println("Máquina: " + maquina);
        System.out.println("Temperatura: " + temperatura + " °C");
        System.out.println("Producción: " + piezas + " piezas");
    }
}
```

Ahora la salida representa valores almacenados en variables. Esto conecta directamente con el próximo capítulo de variables.

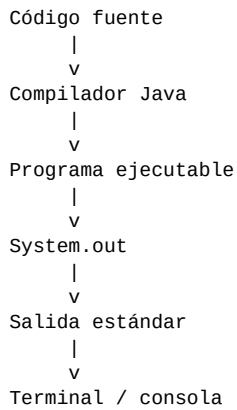
Nivel 4 — Ejemplo profesional

```
public class ReporteProduccion {
    public static void main(String[] args) {
        String maquina = "M-01";
        String estado = "ACTIVA";
        int piezas = 1250;
        double temperatura = 71.38;
        double eficiencia = 94.756;

        System.out.println("=====");
        System.out.println("      REPORTE DE PRODUCCIÓN      ");
        System.out.println("=====");
        System.out.printf("Máquina:      %s\n", maquina);
        System.out.printf("Estado:      %s\n", estado);
        System.out.printf("Piezas:      %d\n", piezas);
        System.out.printf("Temperatura: %.2f °C\n", temperatura);
        System.out.printf("Eficiencia:   %.2f %%\n", eficiencia);
        System.out.println("=====");
    }
}
```

Este ejemplo integra datos, concatenación, `println()`, `printf()`, formato decimal y una estructura visual. En una aplicación real, los datos podrían llegar de sensores, PLC, bases de datos o servicios de red.

11. ¿Qué ocurre cuando ejecutas una salida?



El esquema es una simplificación educativa. Internamente Java utiliza mecanismos de ejecución más complejos, pero sirve para comprender la idea: una instrucción de salida escribe información mediante un flujo asociado a la salida estándar.

Un **flujo** (stream) puede entenderse por ahora como un canal por el que la información se desplaza desde el programa hacia un destino. El concepto será estudiado con mayor profundidad más adelante.

12. Errores comunes al trabajar con salidas

Olvidar ;

Ejemplo: `System.out.println("Hola")`

Por qué: La instrucción queda incompleta.

Corrección: Agregar ;.

Confundir print y println

Ejemplo: `print("A"); print("B");`

Por qué: La información permanece en la misma línea.

Corrección: Elegir `println` cuando se necesite salto.

No cerrar comillas

Ejemplo: `System.out.println(Hola);`

Por qué: La cadena queda abierta y altera el análisis de la línea.

Corrección: Cerrar el texto con `"`.

Escribir mal System.out

Ejemplo: `System.outt.println("Hola");`

Por qué: El nombre escrito no corresponde a un elemento válido.

Corrección: Revisar ortografía y mayúsculas.

Formato incompatible

Ejemplo: `System.out.printf("%d", 12.5);`

Por qué: El especificador no corresponde al valor.

Corrección: Usar un formato compatible.

Confundir suma y concatenación

Ejemplo: "Total: " + 10 + 5

Por qué: Puede resultar 105 en lugar de 15.

Corrección: Agrupar la suma: "Total: " + (10 + 5).

Porcentaje en printf

Ejemplo: System.out.printf("50%");

Por qué: % inicia un especificador de formato.

Corrección: Usar %% para un porcentaje literal.

13. Errores de confusión comunes

Conceptos	Diferencia	Ejemplo
print() / println()	println agrega salto; print no.	print("A"); println("B");
println() / printf()	println es simple; printf controla formato.	println("Temp: "+t) / printf("Temp: %.2f",t)
String / salida	String es tipo de dato; los métodos de salida presentan datos	String n="Ana"; println(n);
\n / %n	Ambos generan salto; %n se usa dentro de printf.	"A\nB" / printf("A%nB")
+ numérico / + concatenación	Depende de los operandos y del orden.	10+5 / "Total: "+10+5

14. Buenas prácticas

Diseña salidas para humanos

Usa etiquetas claras, orden lógico y unidades. '72.5 °C' es mucho más informativo que '72.5'.

Mantén formato consistente

Usa encabezados y separadores de forma uniforme para facilitar lectura y mantenimiento.

Usa printf() cuando el formato importe

Es apropiado para decimales, reportes y datos alineados.

No abusos de comentarios

Los comentarios deben aportar contexto, no repetir literalmente la instrucción.

Prueba la salida

La consola permite verificar tanto el formato como parte del comportamiento del programa.

Separa presentación y lógica al crecer

En sistemas grandes, evita concentrar toda la lógica dentro de una cadena de salida.

Usa nombres descriptivos

Aunque el nombre pertenece al dato, afecta directamente la claridad del mensaje mostrado.

Piensa en mantenimiento

Una salida clara hoy debe seguir siendo entendible cuando el programa crezca.

15. Conectando conocimientos

La salida utiliza directamente la sintaxis aprendida en el capítulo anterior: clases, main, bloques, instrucciones, comillas y punto y coma. Ahora esas estructuras se utilizan para comunicar información.

Conocimiento	Relación con salidas
Sintaxis	Permite escribir correctamente las instrucciones <code>System.out</code> .
Clases y main	Proporcionan el lugar donde se ejecutan las salidas.
Variables	Sus valores pueden mostrarse mediante concatenación o <code>printf()</code> .
Tipos de datos	Influyen en los formatos apropiados.
Entrada de datos	Permitirá recibir información y posteriormente mostrar resultados.
Condicionales	Permitirán mostrar mensajes según estados.
Bucles	Permitirán presentar listas y reportes repetitivos.

16. Ejercicios prácticos

No consultes una solución completa antes de intentarlo. El objetivo es desarrollar la habilidad de construir y corregir tus propias salidas.

Ejercicio 1 — Comprensión básica

Objetivo: Practicar `println()`.

Enunciado: Crea un programa que muestre exactamente tres líneas: CURSO DE JAVA / CAPÍTULO 2 / SALIDAS EN JAVA.

Requisitos:

- Usa clase y main.
- Usa `println()` para cada línea.

Pistas:

- Construye primero la clase.
- Agrega una salida por línea.

Ejercicio 2 — Aplicación

Objetivo: Distinguir `print()` y `println()`.

Enunciado: Construye una salida que muestre 'Máquina: M-01 | Estado: ACTIVA' en una sola línea y después 'Sistema listo' en otra.

Requisitos:

- Usa al menos un `print()`.
- Usa `println()`.

Pistas:

- Piensa cuál método mantiene la siguiente salida en la misma línea.

Ejercicio 3 — Integración

Objetivo: Combinar variables y salida.

Enunciado: Declara nombre de máquina, temperatura y piezas. Después muestra un reporte mediante concatenación.

Requisitos:

- Usa String, double e int.
- Incluye unidades.
- Agrega encabezado.

Pistas:

- Declara primero los datos.
- Después diseña cada mensaje.

Ejercicio 4 — Caso práctico

Objetivo: Crear reporte de producción.

Enunciado: Diseña un reporte con código, estado, piezas, temperatura y velocidad. Usa printf() al menos dos veces.

Requisitos:

- Incluye cinco datos.
- Usa formatos compatibles.
- Temperatura con dos decimales.

Pistas:

- Haz primero una versión simple.
- Después mejora con printf().

Ejercicio 5 — Reto

Objetivo: Diseñar una salida profesional.

Enunciado: Construye el 'Panel de Monitoreo Industrial' con encabezado, fecha simulada, tres máquinas, estado, producción y temperatura.

Requisitos:

- Usa variables.
- Usa separadores.
- Usa printf() en al menos tres valores.
- Incluye unidades.

Pistas:

- Diseña el reporte en papel.
- Divide por secciones.
- Decide los tipos antes de elegir formatos.

17. Mini proyecto del capítulo

Nombre: Panel de monitoreo de una línea de producción

Contexto: una planta necesita una aplicación de consola que presente información básica de sus máquinas. Todavía no habrá conexión real con PLC o sensores; el objetivo es construir la capa inicial de presentación.

Objetivo: construir un reporte claro utilizando todas las técnicas principales de salida estudiadas.

Requisitos funcionales:

- Mostrar encabezado.
- Mostrar tres máquinas.
- Mostrar código, estado, producción, temperatura y velocidad.
- Usar `print()`, `println()` y `printf()` intencionalmente.
- Temperaturas con dos decimales.
- Mostrar unidades.
- Usar separadores.

Requisitos técnicos: Java, clase principal, main, variables básicas, `System.out`, concatenación y `printf()`.

Paso 1

Crea la clase y main. Ejecuta el programa vacío para verificar su estructura.

Paso 2

Declara los datos de la primera máquina. Decide qué valores son texto, enteros o decimales.

Paso 3

Construye una salida con `println()` y concatenación. Después mejora algunas líneas usando `printf()`.

Paso final

Integra las tres máquinas, revisa formato, unidades, decimales y legibilidad.

Pistas: primero consigue que funcione; después mejora la presentación; finalmente revisa consistencia. No intentes resolver todo simultáneamente.

18. Comprueba tus conocimientos

1. ¿Qué significa salida de información en un programa?
2. ¿Qué representa de forma general `System.out`?
3. ¿Cuál es la diferencia entre `print()` y `println()`?
4. ¿Qué ocurre después de ejecutar `println()`?
5. ¿Qué función cumple `+` cuando combina texto y valores?
6. ¿Qué resultado produce `"Total: " + 10 + 5` y por qué?
7. ¿Cómo representas un salto de línea dentro de una cadena?

8. ¿Para qué sirve `\t`?
9. ¿Qué ventaja ofrece `printf()`?
10. ¿Qué significa `%.2f`?
11. ¿Por qué se usa `%%` para mostrar un porcentaje con `printf()`?
12. ¿Qué error existe en `System.out.println("Hola");`?
13. ¿Qué estrategia usarías para presentar un reporte industrial legible?
14. ¿Qué diferencia hay entre `String` y un método de salida?
15. ¿Cómo queda la salida de `print("A"); println("B"); print("C");`?
16. ¿Por qué conviene mostrar unidades como `°C` o `RPM`?

19. Respuestas y explicaciones

1. La salida es la información que el programa comunica hacia un destino; aquí, principalmente la consola.
2. `System.out` representa el acceso habitual a la salida estándar; sobre él se usan métodos como `print`, `println` y `printf`.
3. `print` no agrega salto automático; `println` sí.
4. Escribe el contenido y mueve la siguiente salida a una nueva línea.
5. Permite construir mensajes uniendo texto y valores.
6. Produce 'Total: 105' porque después de aparecer texto, los + siguientes concatenan.
7. Con `\n` dentro de una cadena; dentro de `printf` también puede usarse `%n`.
8. Representa una tabulación.
9. Permite controlar el formato de números y texto.
10. Mostrar un valor decimal con dos posiciones después del punto.
11. Porque `%` inicia un especificador de formato; `%%` representa un porcentaje literal.
12. Falta la comilla de cierre.
13. Separaría por secciones, usaría etiquetas claras, unidades y formato numérico consistente.
14. `String` es un tipo de dato textual; `print`, `println` y `printf` son métodos para presentar información.
15. Queda AB en la primera línea y C en la segunda.
16. Porque un número sin unidad puede ser ambiguo.

20. Resumen del capítulo

Concepto	Para qué sirve	Cuándo usarlo	Error común
<code>System.out</code>	Acceder a salida estándar	Consola	Escribirlo mal
<code>print()</code>	Escribir sin salto	Línea por partes	Esperar salto
<code>println()</code>	Escribir y saltar	Mensajes/reportes	Confundirlo con <code>print</code>
<code>+</code>	Combinar texto y valores	Mensajes dinámicos	105 vs 15
Escape	Representar caracteres especiales	Formato textual	Olvidar <code>\</code>

Concepto	Para qué sirve	Cuándo usarlo	Error común
printf()	Controlar formato	Reportes/números	Formato incompatible

Ideas clave que debes recordar

- La salida permite que un programa comunique información.
- System.out representa la salida estándar habitual.
- print() no agrega salto automáticamente.
- println() agrega salto.
- + puede concatenar texto y valores.
- \n produce un salto de línea.
- \t produce una tabulación.
- printf() controla el formato.
- Los especificadores deben corresponder al valor.
- Los reportes profesionales deben ser claros y consistentes.
- Las salidas son fundamentales para aprender, probar y diagnosticar programas.

21. Mapa de aprendizaje

Progresión	Resultado
Conocimiento previo	Sintaxis, clases, main, bloques y ;.
Nuevo concepto	Salida mediante System.out.
Aplicación básica	print() y println().
Aplicación práctica	Concatenación y secuencias de escape.
Integración	printf(), variables y reportes.
Uso profesional	Salidas legibles y separación entre presentación y lógica.

El siguiente paso natural son **variables y tipos de datos**. Las variables permitirán que las salidas representen información que cambia durante la ejecución. Después, la entrada de datos completará el ciclo fundamental: **entrada** → **procesamiento** → **salida**.

22. Verificación final

- Puedo explicar qué es una salida.
- Puedo explicar System.out.
- Puedo usar print().
- Puedo usar println().
- Entiendo cuándo elegir print o println.
- Puedo concatenar texto y valores.
- Puedo usar \n y \t.

- Puedo usar `printf()`.
- Puedo controlar decimales con `%.2f`.
- Puedo detectar errores de comillas, ; y formatos.
- Puedo construir un reporte para una máquina o línea de producción.
- Comprendo cómo las salidas preparan el camino para variables y entrada.